MCP FOR DEVELOPERS

PAGE 1 : WHAT IS MCP ?

★ 1. What is MCP ?

- MCP (Model Context Protocol) is an open protocol that standardizes how AI applications connect to external tools, data sources, and systems.
- It acts as a universal "bridge" between LLMs and the real world.
- Think of MCP as a "USB-C port" for AI apps
  - one standard, many capabilities.

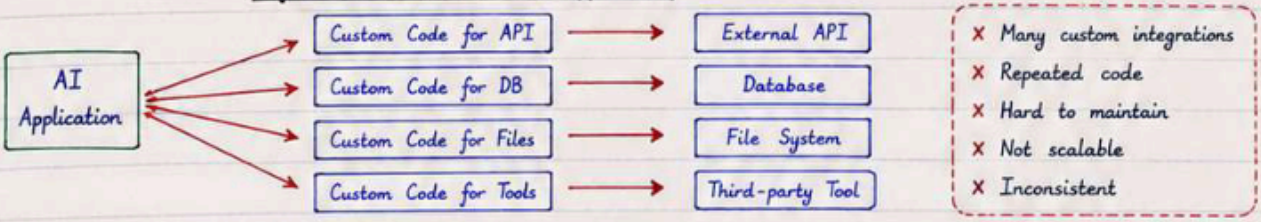
Why MCP was created ?

- Before MCP, every AI app had to build custom integrations for APIs, databases, files, tools, etc.
- This leads to :
  - Too much boilerplate code
  - Inconsistent implementations
  - Hard to maintain and scale
  - Limited reuse of tools
- MCP solves this by providing a standard, secure and reliable way to connect AI models with external capabilities.

2. Problem with Traditional AI Integrations

- Each AI application connects directly to different APIs / databases / tools.
- Every integration is custom-built and tightly coupled.
- No standard way to describe tools, inputs, outputs and capabilities.
- Difficult to share and reuse tools across projects.
- Hard to secure and control access to tools and data.

Before MCP (Traditional Approach)



3. LLM ↔ Tools Problem

- LLMs are great at reasoning and generating text.
- But they cannot directly access real-world tools or data.
- They need a standard way to :
  - understand available tools
  - call tools safely
  - get structured results
  - use that result in the conversation.

4. MCP as a Standard Interface

- MCP provides a standard layer that sits between AI apps and external systems.
- It defines how tools are described, how they are called, and how results are returned.
- Follow "One Protocol, Many Possibilities" approach.

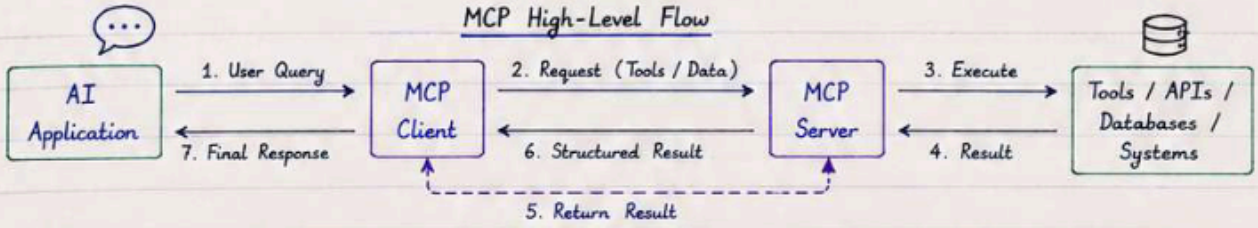
5. MCP vs Direct API Integration

Direct API Integration	MCP Integration
Custom code for every tool	One standard protocol
Hard to reuse	Easy to reuse tools
Inconsistent	Consistent & structured
Hard to secure	Security built-in
Tight coupling	Loose coupling
Difficult to scale	Easy to scale

6. Real-world Examples

- Connect AI to your database to answer questions.
- Let AI search your docs, files or knowledge base.
- Allow AI to create tickets, send emails, or trigger workflows.
- Build AI agents that can perform real actions in your systems.

MCP High-Level Flow



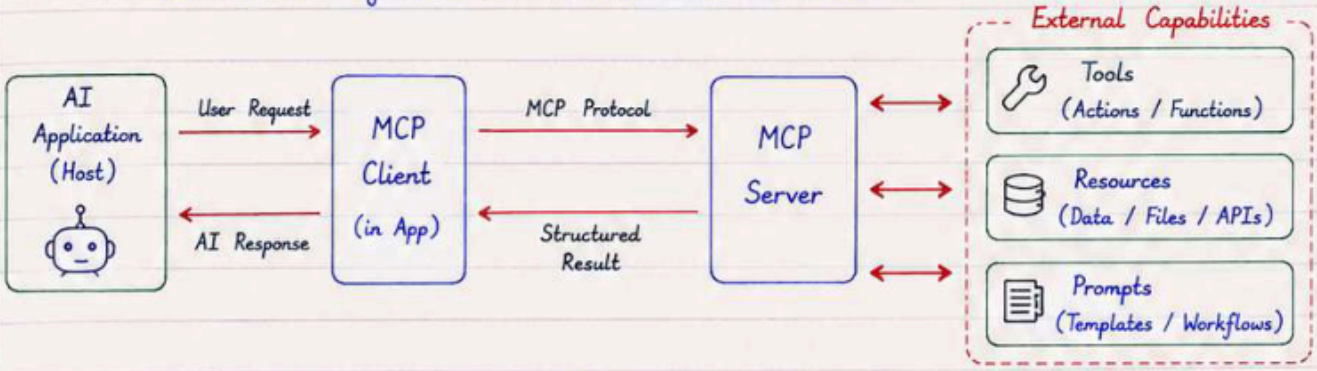
In short : MCP standardizes how AI applications connect to tools, data and systems. It makes AI more powerful, secure, reusable and production-ready.

MCP FOR DEVELOPERS

PAGE 2 : ARCHITECTURE & CORE CONCEPTS

★ 1. MCP Architecture Overview

- MCP defines a standard way for AI applications (hosts / clients) to connect to external tools, data and systems using MCP servers.
- It is transport-agnostic and supports multiple communication methods (stdio, HTTP, etc.).
- The architecture is designed to be modular, secure and extensible.



★ 2. Core Components

- **Host :** The environment where the AI application runs (e.g., VS Code, Web app, CLI tool, Chat interface).
- **MCP Client :** Lives inside the host application. Responsible for connecting to MCP servers, sending requests and receiving results.
- **MCP Server :** A lightweight program that exposes capabilities (tools, resources, prompts) through the MCP protocol.
- **Tools :** Model-controlled actions. Functions that the AI can call to perform real work (e.g., search, fetch data, create issue).
- **Resources :** Application-controlled data sources. Provide information that AI can read from (e.g., files, database records, API responses).
- **Prompts :** Pre-defined templates / workflows that guide the AI to perform tasks in a consistent way.

★ 3. MCP Capabilities (Negotiated)

- When a client connects to a server, they exchange capabilities.
- Both sides know what features are supported.
- This ensures compatibility and safe communication.

Example Capabilities

- ✓ tools/list, tools/call
- ✓ resources/list, resources/read
- ✓ prompts/list, prompts/get
- ✓ logging, progress, sampling
- ✓ authentication, completion, etc.

★ 4. Three Building Blocks of MCP

Tools (Model-Controlled)

Functions that AI can decide to call to take action in the world.

Examples :
- search_users()
- create_ticket()
- get_order_status()

Resources (Application-Controlled)

Data and context that AI can read but cannot modify.

Examples :
- database://customers
- file://project/README.md
- api://docs/list

Prompts (User-Controlled)

Reusable templates that help guide AI for specific tasks.

Examples :
- summarize_document
- analyze_code
- troubleshoot_issue

MCP FOR DEVELOPERS

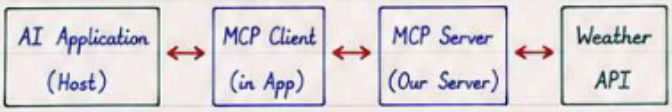
@aman_views

PAGE 3 : YOUR FIRST MCP SERVER (TYPEScript EXAMPLE)

★ 1. What will we build ?

- We will create a simple MCP server that exposes one tool : get_weather(city).
- The AI application (client) can call this tool to get real-time weather information.

Overview Flow



★ 2. Steps to Build MCP Server (TypeScript)

① Create a new Node.js project
② Install MCP SDK for TypeScript
③ Create server instance
④ Register a tool
⑥ Define input schema (arguments)
⑥ Implement tool logic
⑦ Start the server

★ 3. Project Setup

```
# 1. Create project
mkdir weather-mcp-server
cd weather-mcp-server
npm init -y
# 2. Install MCP SDK
npm install @modelcontextprotocol/sdk
npm install zod
# 3. Install TypeScript tools
npm install -D typescript ts-node @types/node
npx tsc --init
```

★ 4. Create MCP Server with One Tool

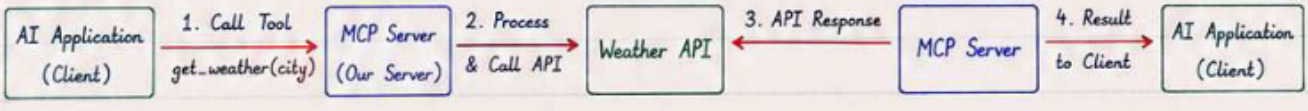
```
import { Server } from "@modelcontextprotocol/sdk/server/index.js";
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";
import { z } from "zod";

// Create MCP Server
const server = new Server(
{
  name: "weather-server",
  version: "1.0.0",
}
);
// Register a Tool
server.tool(
  "get_weather",
  "Get current weather for a given city",
  {
    city: z.string().describe("City name (e.g., Delhi)")
  },
  async ({ city }) => {
    // Call external API (example using open-meteo)
    const url = `https://api.open-meteo.com/v1/forecast?current_weather=true&timezone=auto&${
        city=${city}`;
    const res = await fetch(url);
    const data = await res.json();
    const weather = data.current_weather;
    return {
      content: [
        { type: "text", text: `Weather in ${city}: ${weather.temperature}°C, wind ${weather.windspeed} km/h` }
      ]
    };
  }
);
// Start Server
const transport = new StdioServerTransport();
await server.connect(transport);
console.log("MCP Weather Server is running...");
```

Explanation

- We create a server with name and version.
- We register a tool called get_weather.
- Tool takes one argument: city.
- We call an external API to get weather data.
- We return structured result back to the client.

★ 5. Tool Execution Flow



MCP FOR DEVELOPERS

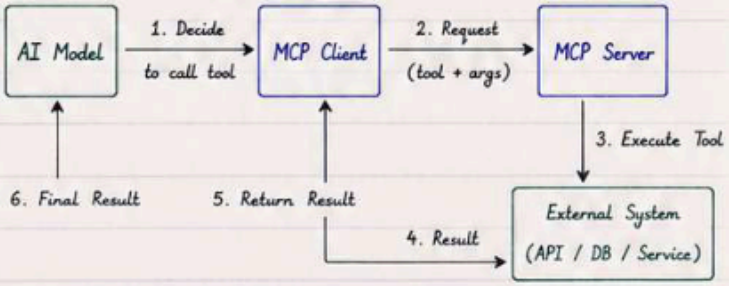
@aman_views

PAGE 4 : MCP TOOLS IN DETAIL

★ 1. What is an MCP Tool ?

- Tools are functions or actions that the AI model can call to perform real work.
- Tools are model-controlled (called by the model).
- Each tool must have a name, description, input schema and output schema.
- MCP server exposes tools and executes them when requested by the client.

Tool Call Flow



★ 2. Anatomy of a Tool

```
{
  "name": "get_user",
  "description": "Get user information by user ID",
  "inputSchema": {
    "type": "object",
    "properties": {
      "userId": { "type": "string", "description": "User ID" }
    },
    "required": ["userId"]
  },
  "outputSchema": {
    "type": "object",
    "properties": {
      "id": { "type": "string" },
      "name": { "type": "string" },
      "email": { "type": "string" },
      "status": { "type": "string" }
    }
  }
}
```

name	Unique name of the tool.
description	What the tool does. Helps the model decide when to use it.
inputSchema	Defines inputs (arguments) the tool accepts.
outputSchema	Defines the structure of the result returned.
required	List of mandatory input fields.
properties	Individual fields with their types and descriptions.

★ 3. Example Tools

Tool Name	Description	Example Use
search_users	Search users by name or email	Find a user in the system
create_ticket	Create a support ticket	Raise an issue for a user
get_order	Get order details by order ID	Track order status
update_order	Update order information	Change order address
send_email	Send an email	Send confirmation email
query_db	Run a database query	Get custom report

★ 4. Tool Execution Example (Flow)

AI Model decides to call tool → Sends tool name + arguments → MCP Server executes the tool → Server returns result → AI Model uses the result

Example : get_user tool

- Model decides to get user details.
- Sends → { "name": "get_user", "arguments": { "userId": "123" } }
- Server calls external API / DB.
- Returns → { "id": "123", "name": "Aman", "email": "aman@gmail.com" }
- Model uses this result in the conversation.

★ 5. Best Practices for Tools

- ✓ Use clear and descriptive names.
- ✓ Write helpful descriptions for the model.
- ✓ Validate all inputs on the server.
- ✓ Handle errors gracefully and return meaningful messages.
- ✓ Keep tools focused - do one thing well.
- ✓ Prefer read-only tools when possible.
- ✓ Log tool usage for monitoring and debugging.

★ 6. Tool Result Types

- **Success Result**
  Return structured data as defined in outputSchema.
- **Error Result**
  Return error object with message and code.
- **Progress / Streaming (optional)**
  For long running tools, send progress updates.

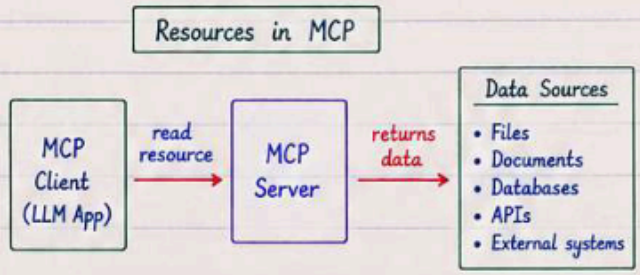
MCP FOR DEVELOPERS

@aman_views

PAGE 5 : RESOURCES & PROMPTS

★ 1. What are Resources?

- Resources are data that the MCP server exposes to the AI application.
- They can be files, documents, database records, API responses, or any other structured data.
- Resources are read-only and controlled by the application (server).
- They help the model get context and information without calling tools.



★ 2. Types of Resources

1. Files : e.g., text files, PDFs, CSV, JSON, markdown files.
2. Documents : e.g., knowledge base, product docs, policies.
3. Database records : e.g., user data, orders, logs.
4. API responses : e.g., data from external APIs.

Resource URI Examples

```
file://docs/guide.md
db://users/123
api://github/repos/aman/views
docs://company/policies
file://config/settings.json
```

★ 3. Defining Resources in MCP (TypeScript)

```ts
import { McpServer, ResourceTemplate } from "@modelcontextprotocol/sdk/server/mcp.js";
import { z } from "zod";

// Create MCP server
const server = new McpServer({
  name: "docs-server",
  version: "1.0.0",
});

// Register a resource
server.resource(
  "read-file",
  "Read a file from the local filesystem",
  { uri: z.string() },
  async (uri) => {
    const content = await fs.readFile(uri, "utf-8");
    return { contents: [{ uri, mimeType: "text/markdown", text: content }] };
  }
);
```

Key Points

- resource() registers a resource with a URI and description.
- You can use ResourceTemplate for dynamic resources.
- Resources are read-only.
- Return data in a structured format (text, json, etc.).
- Can be used by the LLM to provide context for better responses.

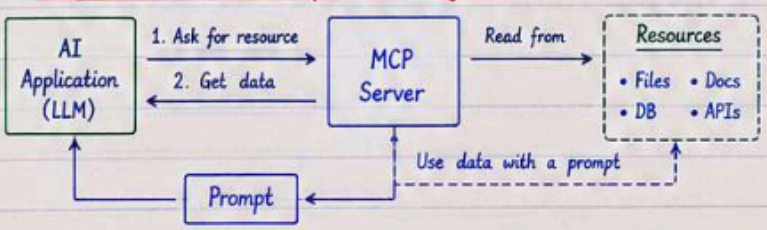
★ 4. Prompts in MCP

- Prompts are reusable templates that help the model interact with your server in a consistent way.
- They can include arguments and create workflows for common tasks.
- Prompts are user-controlled, meaning the user can choose which prompt to use.
- They make it easier to guide the model with predefined instructions.

★ 5. Defining Prompts (TypeScript)

```ts
server.prompt(
  "summarize-doc",
  "Summarize a document into key points",
  {
    uri: z.string().describe("Document URI"),
    maxPoints: z.number().optional().default(5)
  },
  ({ uri, maxPoints }) => {
    return {
      messages: [
        { role: "user", content: `Summarize the document at ${uri}
        into ${maxPoints} key points.` }
      ]
    };
  }
);
```

★ 6. How Resources & Prompts Work Together



Summary

- Resources provide data to the model.
- Prompts provide reusable templates.
- Both are key building blocks of MCP.
- They help the model access information and perform real-world tasks efficiently.
- Use resources for data, and prompts for workflows and guidance.

MCP FOR DEVELOPERS

@aman_views

PAGE 6 : TRANSPORT & COMMUNICATION

1. MCP Communication Basics

- MCP uses JSON-RPC 2.0 for all communication.
- Communication happens over different transports.
- Messages can be Requests, Responses or Notifications.
- Protocol is stateless - each message is independent.

Message Types

Type	Purpose	Example
Request	Client asks server to do something.	tools/call
Response	Server sends result of a request.	Result / Error
Notification	One-way message, no response expected.	progress/update

2. Transports in MCP

A. stdio (Local Communication)

- Used when client and server run on the same machine.
- Simple and fast.
- Good for CLI tools and local integrations.

MCP Client ⟷ stdio (stdin/stdout) ⟷ MCP Server

B. Streamable HTTP (Remote Communication)

- Used for remote servers.
- Communication over HTTP.
- Supports streaming responses.
- Recommended for production.

MCP Client ⟷ HTTP (Streamable) ⟷ MCP Server

Aspect	stdio	Streamable HTTP
Use Case	Local processes	Remote servers
Transport	stdin / stdout	HTTP (Streamable)
Performance	Very fast	Network dependent
Security	Local machine	Use Auth & HTTPS
Best For	CLI tools, local agents	Web apps, cloud, production

3. JSON-RPC Message Structure

Request Example

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "get_user",
    "arguments": {
      "userId": "123"
    }
  }
}
```

Response Example (Success)

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "id": "123",
    "name": "Aman",
    "email": "aman@gmail.com"
  }
}
```

Response Example (Error)

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "error": {
    "code": -32601,
    "message": "Method not found"
  }
}
```

4. Communication Flow Overview

MCP Client (Sends Request) → 1. Request (JSON-RPC) → Transport (stdio / HTTP) → MCP Server (Processes Request) → 2. Execute Logic → MCP Server (Send Response) → 3. Response (JSON-RPC) → MCP Client (Receives Result)

Key Points

- ✓ All messages follow JSON-RPC 2.0.
- ✓ Choose transport based on your use case.
- ✓ Use Streamable HTTP for production.
- ✓ Keep communication secure.
- ✓ Handle errors properly.
- ✓ Design for stateless interactions.

MCP FOR DEVELOPERS

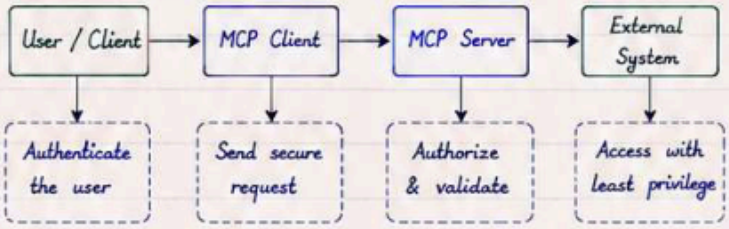
@aman_views

PAGE 7 : SECURITY & PRODUCTION BEST PRACTICES

1. Why Security Matters in MCP ?

- MCP servers can access powerful systems like databases, APIs, files, cloud services.
- If not secured properly, an AI model could perform unwanted or harmful actions.
- We must follow the principle of "Least Privilege".

2. Security Layers in MCP



3. Key Security Practices

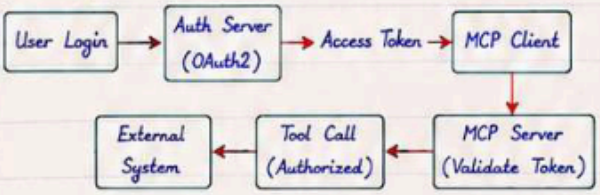
① Authenticate users and clients.
② Authorize every tool based on roles / scopes.
③ Validate all inputs and arguments.
④ Use least privilege for tools and API keys.
⑤ Protect against prompt injection.
⑥ Sanitize data before sending to external systems.
⑦ Log all important actions and errors.
⑧ Use HTTPS for remote communication.
⑨ Rotate secrets and tokens regularly.
⑩ Monitor usage and set rate limits.

Common Threats

- Prompt Injection → Malicious instructions from user trying to misuse tools.
- Tool Abuse → AI uses a tool in unintended or harmful way.
- Data Leakage → Sensitive data exposed through tools or responses.
- Privilege Escalation → Accessing resources beyond allowed scope.
- SSRF / Injection → Exploiting external services via crafted inputs.

4. Authentication & Authorization

- Use OAuth 2.0 / OpenID Connect for user login.
- MCP servers should validate access tokens.
- Pass tokens securely from client to server.
- Check scopes / permissions before executing tools.
- Example Flow :



5. Input Validation & Safe Execution

Always Validate :

- Input types and formats
- Allowed values and ranges
- Length of strings / arrays
- Prevent command injection / path traversal
- Use timeouts for external calls
- Handle errors gracefully.

Tip : Never trust the model or user input. Always validate before using.

6. Production Readiness Checklist

- ✓ Secure authentication & authorization
- ✓ Input validation for all tool arguments
- ✓ Peast privilege access for tools
- ✓ HTTPS / secure transport
- ✓ Rate limiting & throttling
- ✓ Audit logs & monitoring
- ✓ Error handling & retries
- ✓ Secrets management (env variables / vault)
- ✓ Regular security updates & dependency checks
- ✓ Backup and disaster recovery plan

7. Secure MCP System Architecture (Example)

